

```
class Bhagya
{
public static void main(String args[])
{
System.out.println("Welcome to Selenium class");
}
}
```

Above Sample Program Giving instruction to the System that Display Welcome to Selenium Class System Will not understand a Program Which is written in English

It is understandable by only humans that language we called as Highlevel Language

Therefore the task is converting high level language to Low Level Language i.e system understandable language

To get System understandable file We need to compile the Program:

Compiling a Java program means taking the programmer-readable text in your program file (also called source code) and converting it to bytecodes, which are platform-independent instructions for the Java VM.

Command to Compile Program: javac filename.java

A program has to be converted to a form the Java VM can understand so any computer with a Java VM can interpret and run the program.

Once your program successfully compiles into Java bytecodes, you can interpret(translate) and run applications on any Java VM.

Steps to Compile, Execute Program

1. Save Java file (source code)with any name with .java extension
2. Open cmd and navigate to the place where Source code is saved
3. To compile program: javac filename.java
4. During Compilation Will come to know about grammer errors or syntax error
5. If no syntax error will get .class file (system understable file)
6. To Run the Program: java Classname
7. If no semantic errors will get Output otherwise will se logical errors or semantic errors

Explanation:

Class: class is a collection of features and functionalities, or class is blue print or collection of Objects

Rules to write Classname

- Class name must start with alphabets: uppercase valid ,lower case warning
- Should not start with numbers or special characters
- Should not contain spaces

Object:

A Java object is a member (also called an instance) of a Java class. Each object has an identity, a behavior and a state.

The state of an object is stored in fields (variables), while methods (functions) display the object's behaviour. Objects are created at runtime from templates, which are also known as classes.

Public: It is an *Access modifier*, which specifies from where and who can access the method.

Making the *main()* method public makes it globally available. It is made public so that JVM can invoke it from outside the class as it is not present in the current class.

Static: It is a *keyword* which is when associated with a method, makes it a class related method.

Basic Java Program Structure

The `main()` method is static so that JVM can invoke it without instantiating the class. This also saves the unnecessary wastage of memory which would have been used by the object declared only for calling the `main()` method by the JVM.

Void: It is a keyword and used to specify that a method doesn't return anything. As `main()` method doesn't return anything, its return type is `void`. As soon as the `main()` method terminates, the java program terminates too. Hence, it doesn't make any sense to return from `main()` method as JVM can't do anything with the return value of it.

main: It is the name of Java main method. It is the identifier that the JVM looks for as the starting point of the java program. It's not a keyword.

String[] args: It stores Java *command line arguments* and is an array of type `java.lang.String` class. Here, the name of the String array is `args` but it is not fixed and user can use any name in place of it. The java command-line argument is an argument i.e. passed at the time of running the java program.

Note: We can write code above main method or below main method but if it is called inside main method then only it will execute other wise will not execute.

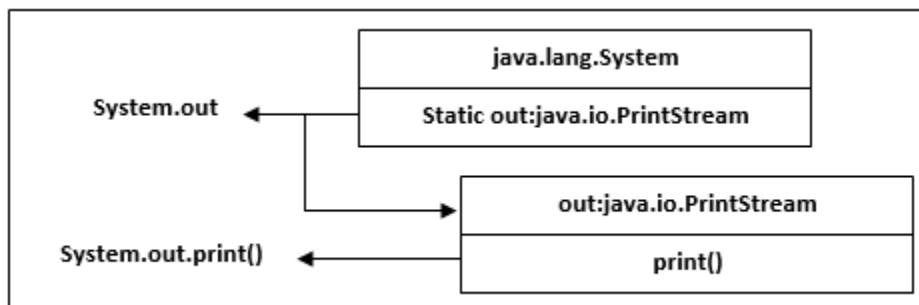
i.e what is written inside main or call inside main method that will only execute.

System.out.println() in Java

In Java, `System.out.println()` is a statement which prints the argument passed to it. The `println()` method display results on the monitor. Usually, a method is invoked by `objectname.methodname()`.

1. `PrintStream obj.print("Hello");`
2. But you cannot create the object to `PrintStream` class directly as above. So, Java provides an alternative way to create the object of `PrintStream` class that is **System.out**.
3. Where **System** is the class name, it is declared as final. The `out` is an instance of the `System` class and is of type `PrintStream`. Its access specifiers are public and final. It is an instance of `java.io.PrintStream`. When we call the member, a `PrintStream` class object creates internally.

1. So, we can call the `print()` method, as `System.out.print()`;
2. It creates the `PrintStream` class object. This object, by default, represents the output device, i.e., the monitor



3.